2022 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 978-1-6654-7956-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICSME55016.2022.00019

Deceiving Deep Neural Networks-Based Binary Code Matching with Adversarial Programs

Wai Kin Wong*, Huaijin Wang*, Pingchuan Ma*, Shuai Wang*, Mingyue Jiang†,
Tsong Yueh Chen‡, Qiyi Tang§, Sen Nie‡ and Shi Wu§
*Hong Kong University of Science and Technology
†Zhejiang Sci-Tech University
‡Swinburne University of Technology
§Tencent Security Keen Lab

***Abstract*—Deep neural networks (DNNs) have achieved a major success in solving challenging tasks such as social networks analysis and image classification. Despite the prosperous development of DNNs, recent research has demonstrated the feasibility of exploiting DNNs using adversarial examples, in which a small distortion is added into the input data to largely mislead prediction of DNNs.**

Determining the similarity of two binary codes is the foundation for many reverse engineering, re-engineering, and security applications. Currently, the majority of binary code matching tools are based on DNNs, the dependability of which has not been completely studied. In this research, we present an attack that perturbs software in executable format to deceive DNN-based binary code matching. Unlike prior attacks which mostly change non-functional code components to generate adversarial programs, our approach proposes the design of several *semantics-preserving* transformations directly toward the control flow graph of binary code, making it particularly effective to deceive DNNs. To speedup the process, we design a framework that leverages gradient- or hill climbing-based optimizations to generate adversarial examples in both white-box and black-box settings. We evaluated our attack against two popular DNN-based binary code matching tools, `asm2vec` and `ncc`, and achieve reasonably high success rates. Our attack toward an industrial-strength DNN-based binary code matching service, `BinaryAI`, shows that the proposed attack can fool remote APIs in challenging black-box settings with a success rate of over 16.2% (on average). Furthermore, we show that the generated adversarial programs can be used to augment robustness of two white-box models, `asm2vec` and `ncc`, reducing the attack success rates by 17.3% and 6.8% while preserving stable, if not better, standard accuracy.

I. Introduction

Software security analysis, such as malware clustering, vulnerability detection, and code plagiarism detection, are the key applications in building today's trust computing infrastructures [73], [63], [42], [7], [67], [6], [11]. From a holistic perspective, these security applications are primarily built on the basis of deciding the *similarity* of two software in executable format. As the paramount component and the trust base in numerous software security applications, Binary code matching (similarity analysis) has enabled the detection of malicious software and promoted cybersecurity professionals' understanding of real-life vulnerability breaches.

Contemporary binary code matching is mostly built on the basis of a "graph view"; by extracting control flow and data flow graphs from binary code, various features can be identified, denoting informative software features that are robust toward syntax-level changes to a certain extent [26], [27]. Most state-of-the-art binary code matching tools are built on the basis of AI methods, particularly deep neural networks (DNNs) [74], [73], [25], [63], [42], [7], [67], [6] to extract high-quality code representations and further compute embeddings from software.

Despite the spectacular development, deep learning models are vulnerable to adversarial examples [29]. Adversarial examples have been demonstrated to bring major threats on the trustworthiness of machine learning (ML) systems by exploiting computer vision (CV) [60], natural language processing (NLP) [19], and various DNN-based applications [61], [58], [76]. Nonetheless, adversarial attacks on DNN-based software matching seldom receive enough attention, yet, once they occur, can cause much confusion to users and be potentially exploited by adversaries. Successful exploitation can break the chain of trust in various software security applications, jeopardizing many real-world scenarios.

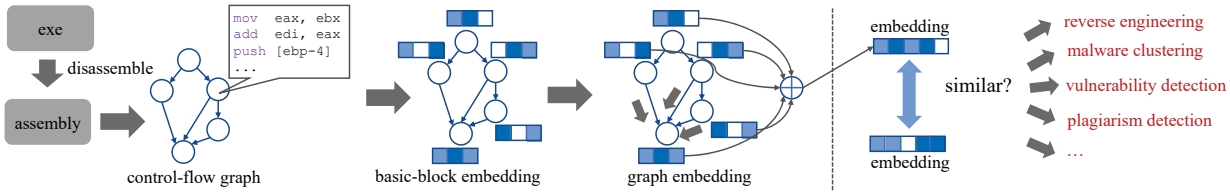
We have observed several attacks generating adversarial software examples against ML models with gradient-based algorithms [35], [36], [56], [13]. Nevertheless, these approaches are highly limited by mutating *only a few byte values in regions that do not affect software execution*. Holistically speaking, software is highly structured and functional, in which randomly mutating data bytes can easily break its functionality. While changing *non-functional* code components easily preserve the software functionality, this indicates unreliability of such methods, perturbations on non-functional components are generally trivial and prone to be detected via security analysis or reverse engineering.

This paper tackles the aforementioned limitations and challenges in existing works. We propose a set of *semantics-preserving* perturbations to extensively mutate the control flow graphs of software samples. Our approach thus becomes more systematic and highly effective to exploit the "graph view" extracted by modern DNN-based binary code matching tools. We design a set of sophisticated strategies to manipulate individual nodes, edges, and the overall control-flow graph representations. In addition, we formulate generating adversarial software examples as an optimization-based procedure and solve it using gradient- or hill climbing-based algorithms for both white-box and black-box settings.

Our extensive engineering efforts enable directly mutating x86 executable files with no source code available. This enables the application scope of our exploitation. Our eval-

Corresponding author is Shuai Wang (shuaiw@cse.ust.hk).

2576-3148/22/$31.00 ©2022 IEEE
DOI 10.1109/ICSME55016.2022.00019



(a) DNN-based software similarity analysis workflow

(b) typical usage scenarios

Fig. 1: DNN-based binary code matching and its enabled software engineering and security downstream applications.

uation on two popular DNN-based executable matching tools, `ncc` [9] and `asm2vec` [25], shows the generated adversarial examples can achieve a high attack success rate, 27.2% for `asm2vec` and 81.0% for `ncc`, with only modest cost. We also exploit a remote industry-strength binary code matching service, `BinaryAI` [73], and achieve an attack success rate of 16.2% (on average). Furthermore, we show that by retraining `asm2vec` and `ncc` with adversarial programs generated by us, the robustness of both models are notably improved, reducing the attack success rates by 17.3% and 6.8% while preserving stable, if not better, accuracy on standard test datasets. This work reveals practical adversarial attack opportunities toward DNN-based software matching tools, promoting the identification of unknown threats before breaches and building more secure DNN systems. In sum, we make the following contributions:

- Binary code matching serves as the cornerstone of many software security applications like vulnerability search, code plagiarism detection, and malware clustering. This work presents a practical and effective adversarial attack toward DNN-based binary code matching. Our work directly mutates program control-flow structures to deceive DNN models.
- Our attack incorporates a set of *semantics-preserving* transformation passes to mutate software. We also incorporate various optimizations and engineering efforts to deliver an effective attacking framework that is applicable toward program executables.
- Evaluations show that our proposed attack deceives modern DNN-based software matching tools in both white-box and black-box settings, making DNN exploitations more practical. Re-training DNN models with our synthesized adversarial programs can notably enhance their robustness without undermining accuracy.

See the codebase of this research at [4].

II. Preliminaries

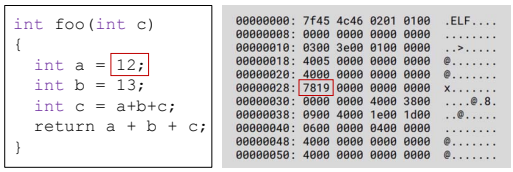
A. DNN-based Binary Code Matching

Software matching compares the similarity (or equivalence) of two given programs with source code or only executable file available. With over twenty years of development, software similarity analysis have been widely used in production to support various software security and comprehension applications. To compare two pieces of software, conventional methods seek to extract features from either syntax-, semantics-, or program structure-level representations. With recent promising advances in representation learning [10], modern matching tools first compute embedding representations from programs, and further predict the similarity of two embeddings using classifiers [73], [63], [42], [7], [67], [6].

Fig. 1(a) presents a holistic viewpoint on how DNN-based binary code matching is conducted. Here we use program executable as the analysis input, but the introduced procedure can be easily adapted to source code. Given an executable `exe` compiled from a program `p`, typical software matching tool first disassembles `exe` into an assembly program and next constructs program control flow graph (CFG) $G = (N, E)$. Each node $n \in N$ denotes a program basic block, which includes a sequence of instructions. Each edge $e \in E$ denotes a control transfer instruction (e.g., a jump instruction). In addition to CFG, some GNN-based analyzers [9] also construct data flow graphs, encoding how variables are used. Typically, natural language embedding models like BERT [24] and PV-DM [40] are first used to convert each basic block into an embedding vector v. The intuition is to treat each basic block as a natural language paragraph, where each instruction deems a sentence.

The graph-level embedding is launched to propagate basic block-level embeddings and aggregate into a graph-level embedding vector [28], [43]. Many graph neural networks (GNNs) have achieved promising results in graph classification tasks, with most GNNs being variants of the message passing neural networks (MPNN) [28]. The standard MPNN implementation has two phases, a message-passing phase and a readout phase. The message-passing phase updates the hidden states at each basic block based on its neighbor nodes. Note that "hidden states" at each basic block are embedding vectors computed before. The message-passing phase can include several iterations, where the extracted numerical representation at each node is gradually updated until reaching a fixed point or a pre-defined threshold. During the readout phase (i.e., aggregation phase), the entire graph feature vector is computed to yield the final embedding output. To date, it has been shown that GNNs, by learning from program control flow and data flow graphs, can extract high-quality software representation that is robust toward program syntax changes [25], [74], [73], [9], [42].

By deciding the similarity of two programs through their embeddings (often requires a downstream classifier trained over embeddings; omitted in Fig. 1), Fig. 1(b) illustrates several enabled security applications. For instance, given a suspicious software, malware analysis is conducted by matching this suspicious sample with malware samples [73], [63], [42], [7]. Similarly, code plagiarism and authorship detection aim to match unknown software with known samples [67], [6], [11]. Overall, higher code similarity indicates the higher chance of detecting a cyber crime.



(a) source code (b) executable file

Fig. 2: Adversarial software perturbation. Software is likely broken by perturbing a single byte in either source code or executable formats.

B. Adversarial Examples in AI Models

This section gives a brief introduction on exploiting AI models with adversarial examples. We then elaborate on previous research focusing on exploiting DNN models.

AE denotes inputs that are minimally perturbed to deceive the prediction of AI algorithms. AE is shown as a general issue for DNN models, and to date, AE attacks have constituted a major threat in various CV and NLP applications. For instance, a number of AE attacks are performed on image classification models with a high success rate [29], [37], [37]: typical attacks aim to engender image misclassification, after applying adversarial perturbations on input images. Adversarial perturbation on images are typically bounded by small L_p-norm ($p \in \{0, 2, \infty\}$); attacks aim to ensure that the perturbations are imperceptible to humans. The perturbation procedure is typically formulated as an optimization-guided procedure [12], taking the gradient changes of the victim model as inputs. Given a target DNN model whose prediction over image x is l, AE attack aims to mutate x into x' such that x' is misclassified into an adversarial-specified label l'. AE often leverages a loss function to quantify the loss to enforce the attacked DNN models for misclassification. Considering the following common form of loss function:

$$Loss(x', l') = \max_{l \neq l'} \{\mathbb{F}_l(x')\} - \mathbb{F}_{l'}(x')$$

where $\mathbb{F}_l(x')$ and $\mathbb{F}_{l'}(x')$ denote the raw, non-normalized, predictions (often referred to as "logits") that the target DNN model generates for label l and l', respectively. When minimizing $Loss$, the target DNN model will misclassify x' into l'.

III. Approach Overview

A. Research Challenge

Recent research synthesizes malware samples and evades DNN-based malware detectors [52], [35], [56]. While optimization approaches like fast gradient method (FGM) are effective to attack image classifiers, their use to adversarial program generation is far from realistic. As in Fig. 2, mutating arbitrary data bytes within a program (either source code or binary code) can impair its functionality and is thus undesirable. To retain the original functionality of a program sample, previously proposed methodologies primarily mutate the non-functional components of the code sample [52], [35], [56], such as by appending a few bytes at the end of the sample or by perturbing data bytes that would never be loaded into memory. It should be accurate to assume that these perturbations can be easily detected via static or dynamic analysis techniques and are therefore not commonly used in practice.

This work performs semantics-preserving graph perturbations over program structures. Our perturbations retain program functionality by design while changing *functional components* of software. We further leverage optimization-based procedures to guide perturbation and rapidly deceive DNNs.

B. Threat Model and Application Scope

Threat Model. We first formulate the threat model and application scope, characterizing the attacker's capability in this research. Overall, we aim to attack DNN-based software matching tools, as software matching serves as the cornerstone of many critical security applications like malware analysis, vulnerability search, and plagiarism detection. Hence, many relevant security tasks can be enhanced by deceiving DNN-based code matching.

We assume that the attacker can have full access to local DNNs (i.e., *white-box* attack), sharing a common assumption in previous adversarial attack papers [35], [36], [56], [13]. Our proposed attack can also be extended into a black-box setting, where we first need to estimate gradients during attack. Our evaluation demonstrates the feasibility and promising results of exploiting both local white-box models and remote black-box services maintained by industry giants.

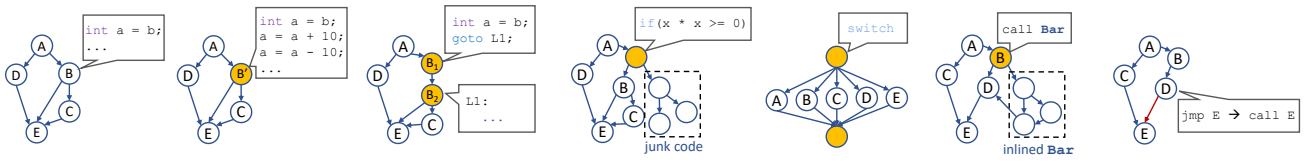
Application Scope. Since most downstream applications of code matching are related to security (e.g., malware analysis), many DNN-based code matching tools accept program executables as the inputs. To deliver a practical tool, we thus decide to implement our attack pipeline toward x86 executable. Moreover, since our proposed attack (see below) does not leverage x86 platform-specific features, the proposed pipeline is applicable to executables on other platforms and also program source code.

C. Semantics-Preserving Graph Perturbation

To tackle the aforementioned limitations and challenges in Sec. III-A, we propose a set of perturbation strategies toward the control-flow graph of executable files. The proposed methods perform effective perturbation toward program control structures to deceive DNNs. More importantly, each perturbation strategy delivers a *semantics-preserving* transformation toward the executable file, in the sense that the transformed output, another piece of executable file, preserve the original (malicious) functionality of the input executable. Soon in Sec. VI, we will show that these semantics-preserving perturbations can be used to effectively exploit DNN-based code matching tools.

Considering a sample program control-flow graph in Fig. 3(a), where each node on the graph denotes one basic block in the program. We first introduce the design of four control flow graph-level perturbation strategies as follows:

Node Rewriting. Our first perturbation performs node-level perturbations to change basic blocks. In particular, we propose a node rewriting scheme which directly extends the content of a basic block by inserting *garbage code*. As explained in Fig. 3(b), such garbage codes denote certain code sequences, which incur no impact on the final computation results. We also perturb the instruction sequences of a particular basic block by replacing a statement with one or a sequence of syntactically different but *semantics-equivalent* statements. For instance, a C statement, `a = a + 10`, can be replaced with



(a) original (b) node rewriting (c) node/edge insertion (d) subgraph insertion (e) graph flattening (f) function inline (g) callsite extension

Fig. 3: Semantics-preserving graph perturbation.

two instructions `a = a + 3` and `a = a + 7` which retains the original computation.

As introduced in Fig. 1, typical DNN-based binary code matching start by embedding each basic block into numerical representations. Hence, we envision by conducting node level rewriting to change the syntactic-level representation of a basic block, the corresponding basic block embedding will be changed as well. Such intuition is also confirmed in our evaluation, as will be reported in Sec. VI.

New Node/Edge Insertion. In addition to rewriting statements within a basic block, we further propose to extend the control flow graph by adding new edges at arbitrary positions. As shown in Fig. 3(c), we "split" a node on the graph by adding a newly-inserted control transfer statement into two adjacent instructions in the node. The inserted control transfer instruction points to the second instruction, thus dividing this target basic block into two blocks.

Inspired by contemporary research in attacking GNNs, which manipulates edges on the graph [18], [22], [68], we propose corresponding manipulation strategies on the program control structures. It is easy to see that the aforementioned node splitting scheme does not change the program semantics, since it simply stitches two adjacent instructions with a newly-inserted control transfer statement. More importantly, the flexible scheme enables inserting edges at arbitrary positions on the program control structure. Our evaluation in Sec. VI also confirms the effectiveness of this scheme to attack DNNs.

Subgraph Insertion. The above "Edge Insertion" scheme splits one node on the control flow graph into two; it thus increments the total number of basic blocks. We next propose a flexible scheme which creates and attaches sub-graphs on arbitrary positions of the control-flow graph *without* changing the program functionality.

Inspired by software obfuscation, we insert an "opaque predicate" associated with an extra collection of nodes [39]. As shown in Fig. 3(d), the so-called opaque predicate inserts a special path condition that is hard to analyze (by code matching engines), but will always be evaluated in one direction (i.e., "true") during runtime. As shown in Fig. 3(d), when creating a new node with opaque predicate which is always evaluated to execute node B (and thus retain the original functionality), we are free to insert arbitrary junk code in the other branch.

Existing work has proposed methods to deceive GNNs by injecting new nodes [58], [59]. Intuitively, by inserting a set of new nodes (referred to as "poisoning nodes" in some literature [58]) into the graph, the attacker primarily changes a set of random walks launched on the affected neighbor nodes and presumably influences computing embeddings. Accordingly, our "semantics-aware" scheme enables to insert arbitrary subgraphs on the program control-flow graph. Guarded by opaque predicates, the inserted subgraph is guaranteed not to disturb the original semantics.

Graph Flattening. In addition to node- and edge-level perturbations, we also propose more comprehensive perturbation strategies toward the program control graph. To this end, we propose to leverage the `switch` statement to convert a given control flow graph to a "flattened" structure. To retain the correct (malicious) functionality, the common practice is to deploy two dispatcher nodes on top and bottom of the flattened control structure to guide the execution flows. As shown in Fig. 3(e), the flattened structure redirects the execution flow to the dispatcher blocks, which uses a global variable to decide which block to further jump to.

The aforementioned graph flattening strategy preserves the original functionality. More importantly, we note that the largely changed control flow structures shall effectively change the original message propagation (see Fig. 1) launched on the control flow graph, presumably changing the information aggregation outputs. Indeed, our evaluation in Sec. VI demonstrates the strength of this method, by achieving a high success rate of exploiting DNN models.

Call Graph Manipulation. We note that the aforementioned graph-level manipulations refer to control flow graphs of functions. In other words, applying each perturbation will change *one* function. We note that this is a reasonable setting, since many modern code matching tools are performing function-level matching [73], [63], [42], [7], [67], [6], [11]. However, since programs usually contain multiple functions, we propose two holistic mutations inspired by compiler optimizations.

Function Inline We first propose a strategy to extend a control flow graph of a function using its callee functions. As shown in Fig. 3(f), we identify all n callsites of function f and inline f to its callsites. This way, we extend the control flow graphs of functions which have these n callsites with the control flow graph of f. This scheme helps enlarge a considerable number of control flow graphs in the mutated executable without breaking the original functionality.

Callsite Extension In contrast to the function inline scheme which merges the control flow graph of a function with its callee, Fig. 3(g) further proposes a scheme named callsite extension. This scheme manipulates a control flow graph of a function and creates new call graph edges, by converting a control transfer statement (e.g., `jmp`, `jz` in x86 assembly code) to a function call. Again, this scheme effectively changes the control flow graph without disturbing the functionality.

These two call graph manipulation schemes, by either enlarging or splitting the control flow graph, shall induce many changes to the results of message aggregation on the graph, presumably changing the embedding outputs. We report to observe consistent results in our evaluation (Sec. VI).

Algorithm 1 White-box attack. As clarified in Sec. VI, each time we mutate an assembly function f.

```
1: function WBAttack(G, N, f)
2:     i ← 0
3:     f̂ ← RandomMutate(f)
                ▷ randomly pick a mutation method to mutate f.
4:     while Match(G, f, f̂) and i < N do
         ▷ "Match" check if we have deceived G; setup in Sec. VI.
5:         v̂ ← Embedding(G, f̂)
6:         g_f ← ∂L(f̂)/∂v̂
7:         t ← PickMutation();
8:         f̂* ← Mutate(f̂, t)
9:         v̂* ← Embedding(f̂*)
10:        δ_f ← v̂* − v̂
11:        if g_f · δ_f > 0 then
12:            f̂ ← f̂*
13:        i ← i + 1
14:    return f̂
```

Ten Mutation Instances. Each transformation complicates the program structures to a certain extent, and different transformations may achieve a *synergistic effect* by iteratively mutating a program, where the output of each iteration (a perturbed executable), will serve as the input of the next iteration. Note that in accordance to the above six perturbation schemes, we have indeed implemented *ten* mutation passes, because some mutation schemes, e.g., node/edge rewriting, can have two instances by either replacing existing instructions or inserting garbage instructions. As a result, the total ten mutation passes in our framework create a large search space, 10^n, where n represents the number of iterations applied toward the input. Inspired by relevant research in this field [35], [36], [52], this research formulates searching the optimal perturbation sequence as an optimization procedure, where statistics methods can help to explore the search space and find optimal sequences promptly (see Sec. IV).

Alternative Solutions. Besides perturbation methods introduced above, we envision proposing other perturbations. For instance, another node rewriting scheme can be proposed to mutate the order of those statements whose relative orders are *not* important in a node on the control-flow graph. Suppose a node consists of three statements `s=a+b; t=a*b; s=s*t`, it is easy to see that the relative order of two statements `s=a+b` and `t=a*b` is not important. We can thus rewrite the node into `t=a*b; s=a+b; s=s*t` without disturbing the semantics. Nevertheless, conducting such statement perturbation requires performing *data dependency analysis*, which is nontrivial for x86 assembly code. The underlying instrumentation platform, Uroboros [47], does not facilitate such data flow analysis utilities. We leave it as one future work to extend the underlying reverse engineering infrastructure to support more perturbations.

IV. Framework Design

Alg. 1 elaborates on the white-box attack setting, which is generally aligned with relevant works in this field (e.g., [53]). The input of our framework is a piece of executable e (source code can be compiled into executable before processing). Given a target DNN model G, we start by disassembling e and recovering all functions (see Sec. V for reverse engineering details). As will be clarified in Sec. VI, most de facto DNN-based code matching tools, including three models evaluated in this work, perform assembly *function-level* matching. Therefore, Alg. 1 accepts an assembly function f as its input and performs mutation over f. However, the proposed method is generally applicable to either assembly functions or the entire program (for the latter case, we need to extend Alg. 1 by iterating each function). Our judgment about "attack success" (line 4) is also toward this particular function f; see details in Sec. VI.

We pre-process function f by transforming f with a randomly selected mutation method (line 3). Generating adversarial examples is formulated as an iterative process (lines 4–13). We re-apply this process to the already mutated function $\hat{f}$ until either the perturbed function successfully deceives G or we have reached a predefined threshold N (line 4). For each iteration, we start by computing the graph embedding vector $\hat{v}$ of $\hat{f}$ (line 5). We then compute the corresponding gradient, g_f, with respect to the DNN's loss function (line 6). The loss function we use is aforementioned in Sec. II-B. Then, we apply a randomly-picked perturbation t on $\hat{f}$ (lines 7–8). We then generate the graph embedding vector $\hat{v}^*$ after applying the attempted mutation (line 9). Given the graph embedding vector generated before and after the current iteration of perturbation (i.e., $\hat{v}$ and $\hat{v}^*$), we compute the difference between $\hat{v}$ and $\hat{v}^*$ (line 10), and decide whether the cosine similarity between g_f and δ_f is positive (line 11). A positive $g_f \cdot \delta_f$ indicates that we are targeting on promising mutations that progressively undermines G. We retain this perturbation and proceed to the next iteration (line 12).

Black-Box Setting. We also consider the black-box setting where DNN models are deployed on a remote host. In black-box settings, we assume to only access the prediction result and its associated confidence score, but are unable to leverage the computed graph embedding vectors (e.g., $\hat{v}$ in Alg. 1). For this setting, we design an exploitation procedure that is derived from the standard hill-climbing approach [55]. The white-box attack procedure depicted in Alg. 1 can be easily extended for black-box settings: while the white-box attack leverages a gradient-based guideline to decide whether retaining a perturbation (line 12 in Alg. 1), we instead accept a perturbation if the returned confidence score (typically a floating point value ranging from 0 to 1) decreases.

V. Implementation

As introduced in Sec. II, to generate adversarial programs, we need to first disassemble the input executable. Recent advances in binary code disassembling has achieved prosperous progress in practice and to realize fool-proof disassembly of real-world software [65], [62], [8]. Indeed, as will be shown in Sec. VI-D, all reverse engineering conducted in this research can correctly disassemble and perturb executable samples used for evaluation.

We use a reverse engineering platform, Uroboros [47], to perturb executables. Uroboros features the so-called "reassembleable disassembling" [65], which enables fool-proof binary decompiling, instrumenting, and further recompiling the instrumented assembly code into another executable. To clarify, the key technique of Uroboros handles address relocation,

a common challenge in binary rewriting. That is, Uroboros will identify all assembly memory addresses in disassembled code, and symbolize those concrete addresses into symbols. This way, binary addresses are correctly handled (since they have been lifted into relocatable symbols). Dynamic jumping in typical x86 assembly can also be correctly handled, given the addresses that may be used by dynamic jumping (e.g., `jmp eax`) can be identified by Uroboros and correctly lifted into relocatable symbols. We have evaluated the semantics correctness of our perturbed binary code, and we report that all perturbed binary code are functional correct (we use the test cases shipped by `coreutils` for testing); see details in Sec. VI-D.

We augment the existing platform of Uroboros with about 1,500 lines of Python code to develop our framework. We also note that the underlying binary code instrumentation platform is *orthogonal* to the proposed attack. We use Uroboros as a proof-of-concept platform for this research. If needed, users can use other (commercial) revese engineering platforms like angr [54] or IDA-Pro [30]. We have released our codebase at [4]. Sec. III-C has introduced schemes implemented in our framework to perturb the control flow of binary code. We list the full implementation details (particularly on x86 assembly) of each proposed scheme at [2]. Also, though we implement our approach to directly translating x86 executable, our schemes are adaptive for other languages or architectures (e.g., ARM64) can be easily constructed in the same way.

VI. Evaluation

Models. We use two local DNN models, `asm2vec` [25], and `ncc` [9], and one remote black-box model, `BinaryAI` [73], to benchmark the present attack. `asm2vec` computes executable embeddings using an extended PV-DM language embedding model [40] and further use random walk to extract graph features. `ncc` generates code embeddings by constructing a contextual flow graph, which subsumes both data flow and control flow features. It then uses DNN-based embedding models to extract a numerical representation.

`BinaryAI`, whose core techniques are published at [73], [74], performs assembly function embedding by first embedding basic blocks with BERT [24], and then conduct graph-level embedding with MPNN [28]. `BinaryAI` provides APIs to compute binary code embedding using its pre-trained model. We leverage its APIs (https://github.com/binaryai) to benchmark the black-box attack capability of our presented technique. It is stated that `BinaryAI` is pre-trained over millions of binary samples [73]. Our observation shows that `BinaryAI` exhibits sufficiently high accuracy which is comparable to its paper [73]. To our best knowledge, `BinaryAI` denotes the state-of-the-art DNN-based software similarity analysis platform, which has been evaluated [73] to outperform other popular DNN-based frameworks or baseline models like Gemini [72]. Another recent work, PalmTree [41], only provides embedding models for assembly instructions and does not release model for function embedding. We tentatively evaluate PalmTree by bridging its instruction embedding with one popular GNN model, GGNN, to compute each function's embedding [43]. However, the result seems not promising, and we decide to not include PalmTree in evaluation.

Evaluation Setup. Most modern code matching tools, including three DNNs evaluated in this research, perform *function-level* analysis. Therefore, we form a dataset of assembly functions as the evaluation dataset (see **Dataset** below).

The standard usage of all three DNN models forms an information retrieval procedure, where given target assembly function f_t and a repository of assembly functions RP, these DNN models retrieve the top-k functions $f_t \in RP$ ranked by their matching score with f_t. Considering a popular downstream application — malware analysis, it is easy to see that in that scenario, the target function f_t denotes a suspicious program, while the repository RP denotes a database of known malware samples. Thus, given a function $f \in RP$, the DNN evading rate can be assessed by showing whether an adversarial example f' derived from f can manifest reasonably low matching score when using f' as the target to query RP.

As will be introduced in **Dataset**, we deduplicate assembly functions in `coreutils` programs into a collection of 1,425 functions. These 1,425 functions, referred to as S_{func}, will be used to form the repository RP for three DNN-based tools.

Dataset. Consistent with most research in this field, we form our evaluation dataset from GNU `coreutils` (version 8.28), a software set commonly used in this line of work. This dataset consists of 105 programs which are the common utility software on Linux platforms. We remove programs in `coreutils` with relatively simple functionality (e.g, `true` and `false`), resulting in a set of 90 programs. To prepare executable files, we compile all programs as x86 executable on Ubuntu 18.04 using `gcc` compiler.

While each `coreutils` program has distinct functionality, we clarify that `coreutils` programs are compiled with a large `coreutils` library which provides common utilities. That is, `coreutils` programs indeed share a number of generally identical functions. To present a fair and in-depth evaluation, we first collect all functions from the executable files of those 90 `coreutils` programs, and deduplicate functions by checking their function names. We also exclude `main` functions and functions with less than three basic blocks as DNN models are seen to have major challenge to analyze such trivial functions. The remaining larger functions with distinct names, in total 1,425, are used in the evaluation. To ease the presentation, this collection of 1,425 functions are referred to as S_{func} in the rest of this paper.

Model Training. Our learning and testing were conducted on a server machine with two Intel Xeon E5-2683 v4 CPUs at 2.40 GHz with 256 GB of memory and two Nvidia 2080 GPU cards. The machine runs Ubuntu 18.04. We report it takes about 2,560 CPU hours (80 hours $\times$ 32 cores) for training and attacking three targeted models.

Deciding Attack Success of `asm2vec`. To define the attack success of `asm2vec`, we first cross-compare every function $f \in S_{func}$. We report that when cross-compare every pair of functions within S_{func} with itself in the pre-trained model, the average matching score yielded by `asm2vec` is 41.4%. Hence, we deem an attack as a success, in case we generate an adversarial program sample reducing the matching score below 41.4% when compare it with its original version. In other words, when the adversarial example has a matching score below 0.41 compared with its original version, analysts

TABLE I: Attack result summary.

	asm2vec	ncc	BinaryAI top-10	BinaryAI top-50	BinaryAI top-100
#Tested functions	293	1276	972	991	996
#Successful attacks	75	755	138	100	79
#Erroneous cases	17	344	332	334	336
Success rate	27.2%	81.0%	21.6%	15.2%	12.0%

would reasonably treat them as "different" and presumably neglect this case.

We emphasize that to present a more challenging setting, we only perturb a function f in case asm2vec yields over 80% similarity score by comparing f to itself. This would retain 293 functions out of 1,425 functions in S_{func}.

Deciding Attack Success of ncc. Similarly, when benchmarking the exploitation of ncc, we first compute the average similarity score of ncc. We report that when cross comparing two functions within S_{func}, the average matching rate yielded by ncc is 54.5%. To present a fair evaluation and conservatively assess the attack success rate, we deem an attack as success, only if the adversarially-perturbed function f' decreases the similarity score to less than 40%, when we compare f' with its original function $f \in RP$.

Deciding Attack Success of BinaryAI. Our tentative study on the BinaryAI shows that most matchings within top-100 manifest very high similarity rate. Therefore, averaging matching scores of function pairs can induce a high threshold T. Envisioning the potential "unfairness" to BinaryAI, we instead use top-N as a threshold. That is, we deem an attack successful, in case the true matching is out of top-N (where N is the threshold quantifying the attack difficulty). For the current evaluation, we use three settings, where N is 10, 50, and 100. Overall, smaller N implies a lower challenge for the attacker, while a larger N (e.g., 100) requires an attacker to deceive BinaryAI significantly and reduce the matching score largely.

As shown in Table I, we report that out of in total 1,425 functions in S_{func}, we need to exclude 429 functions since they are already out of top-100 when being matched to itself. The remaining 996 functions in S_{func} form the repository RP for matching. Similarly, for the $N = 10$ and $N = 50$ settings, the remaining functions are 972 and 991, respectively.

TABLE II: Mutation distribution in successful attacks toward asm2vec.

Perturbation	Counts	Perturbation	Counts
node rewriting$_1$	76	**node rewriting$_2$**	52
node/edge insertion$_1$	77	**node/edge insertion$_2$**	17
node/edge insertion$_3$	52	**subgraph insertion$_1$**	80
subgraph insertion$_2$	60	**function inline**	36
graph manipulation	25	**callsite extension**	44

A. Attacking asm2vec

As reported in Table I, the attack successful rate toward asm2vec is 27.2%. During the exploitation, we find that 17 cases triggering processing errors of asm2vec. Among all success attacks, 68.0% cases require no more than eight iteration of perturbations, while 90% of the successful attacks take no more than sixteen perturbations. The longest perturbation sequence for an input is 20. As clarified in the **Ten Mutation Instances** paragraph in Sec. III, we implement in total ten mutation passes. Table II presents the distribution

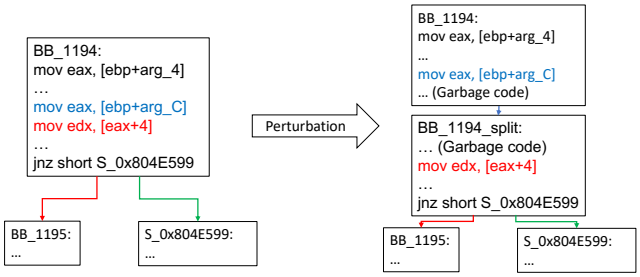


Fig. 4: Case study of a successful attack toward asm2vec.

of involved ten mutation passes in successful attacks toward asm2vec. For instance, the "node rewriting" scheme contains two instances which either replace existing instructions or inserting garbage instructions. See [4] for our codebase and full details of each pass. Overall, Table II implies that all the proposed schemes are highly useful. We deem the result as reasonable: as stated in [25], asm2vec leverages random walk to progressively expand and convert program control flow graphs into embeddings. Applying all of our mutation passes can effectively change the graph structures at different granularities, creating significant difference in the induced vector representations. We note that similar observations (i.e., mutation passes are distributed roughly equally) are obtained from testing two other DNN-based matching tools, as noted below in Table IV and Table III.

Fig. 4 presents a successful case by first applying the node/edge insertion scheme and then applying the node rewriting scheme. An extra jmp statement is inserted to split the block BB_1194 in the left CFG into two basic blocks BB_1194 and BB_1194_split in the right CFG. While merely splitting the basic block does not evade asm2vec, node rewriting scheme toward BB_1194_split introduces extra garbage code, and effectively changes the block-level embedding. Indeed, for this case our tool transforms eight basic blocks similarly to Fig. 4, whose induced changes effectively deceive asm2vec.

While the attack success rate of asm2vec is relatively low compared with ncc (see Table I), we interpret the result as overall promising. Recall to present a mostly challenging setting in benchmarking our technique, we only perturb 293 challenging functions f whose similarity rates are over 80.0% when compared to themselves. In other words, achieving a successful attack requires to largely mutate f to reduce the similarity score from over 80.0% to below 40.0%. Indeed, our manual study shows that such challenging functions usually have unique code patterns that distinguish itself from other functions in the repository RP of 293 functions.

B. Attacking ncc

ncc computes embeddings over LLVM IR code, and to benchmark our dataset of executables, we use a popular binary lifter, retdec [38], to first lift the input executable into LLVM IR programs. As reported in Table I, when attacking ncc, 344 functions in S_{func} lead to errors when we lift their corresponding binary code into LLVM IR with retdec. We obtained 81% success rate with the remaining set of 1,080 functions. We report that 71% cases take less than eight

TABLE III: Perturbation distribution in successful attack toward ncc.

Scheme	Counts	Scheme	Counts
callsite extension	425	**node/edge insertion$_3$**	646
function inline	738	**subgraph insertion$_1$**	392
graph manipulation	702	**subgraph insertion$_2$**	417
node/edge insertion$_1$	632	**node rewriting$_1$**	775
node/edge insertion$_2$	230	**node rewriting$_2$**	594

TABLE IV: Mutation distribution in successful attacks toward BinaryAI.

Scheme	BinaryAI top-10	BinaryAI top-50	BinaryAI top-100
callsite extension	20	16	15
Function Inline	92	75	60
graph manipulation	18	16	15
node/edge insertion$_1$	95	74	62
node/edge insertion$_2$	26	22	17
node/edge insertion$_3$	67	58	48
subgraph insertion$_1$	115	92	72
subgraph insertion$_2$	110	93	77
node rewriting$_1$	87	75	66
node rewriting$_2$	92	74	58

iterations of perturbation to generate adversarial examples that can successfully deceive ncc. 90% of successful adversarial samples are generated within thirteen iterations. Our evaluation in Table I implies that ncc is more "vulnerable" than the other two models. Recall as introduced earlier in this section, ncc builds code embeddings by learning from both program data flow and control flow graphs. While our proposed mutation methods are demonstrated by altering the control flow graphs in Fig. 3, it is easy to see that mutation methods like subgraph insertion can largely complicate the data flow as well, thus notably deceiving ncc.

Table III reports the distribution of applied mutation passes for a successful attack toward ncc. Overall, we interpret the distribution is generally consistent with our findings in attacking asm2vec (see Table II), such that all ten mutation passes manifest non-trivial contribution to deceive the model.

C. Attacking BinaryAI

This section reports the evaluation results of attacking BinaryAI. As mentioned in Sec. IV, we use hill-climbing algorithms to support attacking black-box models. Table I reports the attack success rates regarding three settings, namely $N = 10$, $N = 50$ and $N = 100$. To understand the "# of tested functions" (the second row in Table I), for the $N = 100$ setting, we report that there are 996 (out of 1,425) functions within top-100 ranked list. BinaryAI uses a commercial decompiler, IDA-Pro, as its front end. We note that there are 336 functions IDA failed to decompile after applied any of the perturbation scheme available. We thus achieve a success rate $\frac{79}{997-336} = 12.0\%$. Consistent with our intuition, our attack success rate slightly decreases when N increases, as larger N denotes a more challenging setting for the attacker. Overall, we find that BinaryAI, though manifesting high accuracy on hold-out testing datasets, is still vulnerable under our attack. We find among all success attacks, about 80.4% cases require less than eight iterations of perturbation, which is similar with the two white-box models.

We presented the distribution of mutation passes for all successful attacks in Table IV. We find that two methods that holistically change CFG structure, graph manipulation and callsite extension, are not as effective as others. Though

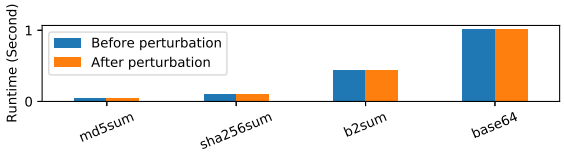


Fig. 5: Runtime before and after perturbation. We note that the performance increase after perturbation is overall negligible.

BinaryAI is a black-box, it is disclosed that this executable matching tool first uses the commercial reverse engineering tool, IDA-Pro [30], to convert the input executable file into its intermediate language named Microcode [3]. In short, this Microcode representation is designed to be resilient toward code mutation; our tentative exploration of IDA-Pro also reveals that program CFGs are "normalized" to certain extent in the Microcode-level representation. This shall explain the relatively low effectiveness of callsite extension and graph manipulation in Table IV.

D. Cost and Correctness Evaluation

Perturbation Correctness. This section reports the cost and correctness of the perturbed executables. As introduced in Sec. III, our perturbation passes are semantics-preserving, meaning that any perturbing should not change the program functionality. To confirm that, we test all the perturbed programs with the shipped test cases in GNU coreutils which are highly comprehensive. We report that **all** perturbed programs can pass the functionality test, confirming that our perturbation does not turn any code into mal-functional.

Perturbation Cost. More importantly, all perturbations introduce new instructions in the programs, which could impose additional performance penalty. We use a popular performance analysis tool on the Linux ecosystem, perf, to benchmark the cost of the perturbed adversarial examples. We use the shipped test cases of coreutils as the benchmark inputs. Overall, we report that for most coreutils binaries, our perturbation only incurs *negligible* performance slowdown (less than 1%). At this step, we select four relatively costly programs performing crypto hash computation to illustrate the induced cost. The results are presented in Fig. 5. We report that the average performance cost is less than 0.36%, showing the extra runtime cost is not a primary concern for our research. We interpret the encouraging findings about cost are due to: 1) many attacks successfully evade DNNs using a small number of perturbations, resulting in low performance penalty, and 2) the mutation can occur on *cold paths*, thus causing negligible effect on runtime overhead. We deem this revealing the strength of our proposed attack.

E. Stealth Evaluation

Stealth deems a critical property in this research [21], indicating the difficulty to recognize an adversarial example. Inspired by relevant research [45], [70], we assess the stealth of generated adversarial examples by measuring whether the present techniques can introduce any *statistical characters* that are inconsistent with normal executable files; such abnormal statistical characters could be leveraged to pinpoint adversarial examples. To this end, we adopt a popular assessment named instruction distribution [48], [14], which measures the ratio of

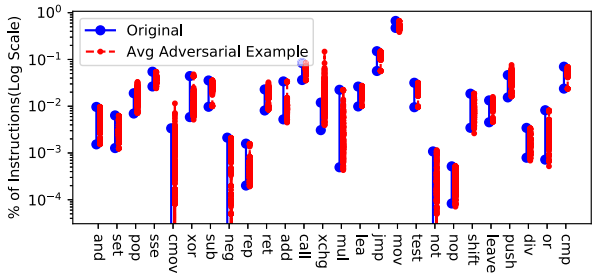


Fig. 6: Instruction distributions in normal executables and adversarial examples.

different types of instructions. We group all x86 instructions into 26 categories and calculate the distribution.

Fig. 6 reports the distribution of instruction types, comparing the original test cases and our generated adversarial examples. For each instruction category, "Original" shows the range of distributions in the original programs, while the red dots denote the distribution of a particular instruction category within an adversarial example. From a holistic perspective, adversarial examples manifest mostly *consistent* distribution with the original programs. Thus, we interpret that adversarial examples are stealthy enough and it is generally difficult, if at all possible, to distinguish adversarial examples with normal executable files. An increase of `xchg` instructions is found in the adversarial examples. Our current implementation uses `xchg` instructions as the "garbage code" to rewrite nodes on control flow graphs. Other garbage code patterns [39] can be used to calibrate the portion of `xchg` instructions.

TABLE V: Ablation: comparing the number of successful attacks under the random-mutation setting vs. guided mutation setting.

	`asm2vec`	`ncc`	`BinaryAI`
Random	51	742	130
Guided	75	755	138

TABLE VI: Ablation evaluation: comparing the average length of mutation iterations in successful attacks under the random-mutation setting vs. guided mutation setting.

	`asm2vec`	`ncc`	`BinaryAI`
Random	5.5	7.3	5.2
Guided	6.9	6.4	5.1

F. Ablation Evaluation

We launch ablation evaluation to answer the question that whether our guided mutation (Sec. IV) are more efficient than random mutation. Recall for `asm2vec` and `ncc` (the "white-box" setting), we use gradient to guide the mutation, while for `BinaryAI` (the "black-box" setting), we use hill-climbing algorithm as the guidance. At this step, we setup a "random mutation" for comparison, where for each iteration step, we randomly select one mutation pass from our ten mutation pass candidates. We compare the average number of mutation iterations launched for both guided and random settings.

We present evaluation results in Table V and Table VI. From Table V, we find that starting from the same number of seeds, guided mutations can find more mutated inputs that successfully deceive DNN models in both white-box and black-box settings. Table VI reveals that the guided mutations in both white-box and black-box settings notably enhances the efficiency of our attack; it helps to successfully deceive DNN models with less number of mutation iterations. The only outlier is `asm2vec`, where random mutation can identify shorter sequences (5.5 vs. 6.9) for successful attacks. Note that as shown in Table V, under guidance mutation, we find 24 more $(75 - 51)$ successful attacks. Our further investigation shows that among these successful attacks, the mutation sequences are usually much longer (ranging from average 16 to 20). These long mutation sequences, whose discovery requires much effort, prolong the average length. Nevertheless, random mutation simply failed to find such long attack sequences, resulting in less number of successful attacks.

Overall, given that a considerable number of attacks can be finished within only a few iterations of mutation, random mutation manifest reasonable effectiveness in certain cases. In summary, we interpret that our guided mutation strategies are generally more effective by finding more and shorter mutation sequences to deceive DNN models in most cases.

TABLE VII: Evaluating augmented models.

	`asm2vec`			`ncc`		
	0	0.5	1.0	0	0.5	1.0
Attack Success	27.2%	25.5%	22.5%	81.0%	79.6%	75.8%
Standard Accuracy	19.1%	17.1%	18.8%	90.9%	91.7%	91.2%

G. Model Augmentation

Data augmentation creates transformations over existing training data samples to enrich model training data. It has been demonstrated that data augmentation improves model performance after retraining on the extended data, and it has been frequently employed as a routine technique to enhance NLP and CV models [75], [69]. Existing approaches, however, are *not* directly applicable to enhance DNN-based software matching tools. Due to the synthetic and semantic constraints of software, arbitrary transformations can easily break a well-formed program. On the other hand, our testing pipeline has generated a large volume of mutated executables for use.

Table VII reports the augmentation results over two white-box models, `asm2vec` and `ncc`. For `BinaryAI`, we are not able to augment it since we don't have access to its models. In Table VII, the "0" columns (the 2nd and 5th columns) denote the evaluation results over the original model. We report the attack success rates (directly taken from Table I). We also present their accuracy on standard test dataset. To measure "standard accuracy" of `ncc`, we use its shipped standard test dataset. As for `asm2vec`, we compute the top-1 accuracy over our formed dataset S_{func} of 293 assembly functions following the standard ten-fold dataset splitting strategy. While the reported top-1 accuracy in Table VII is slightly lower than other relevant research (we interpret the main reason as the lack of training data), we clarify that top-3 and top-5 accuracy of `asm2vec` (omitted in Table VII) is close to data reported by related works [31].

Our testing pipeline generates a large number of mutated software samples. We first randomly select a collection of mutated samples, which equals to $x\times$ size of the model's standard training data ($x \in \{0.5, 1.0\}$, as in Table VII). We

then use these two extended training datasets to train two `asm2vec` models and two `ncc` models, respectively. Next, we re-run our attack pipeline and record the attack success rates and also benchmark the model accuracy on standard test datasets. Note that when re-running the attack pipeline, we generate *different* mutated executables instead of re-using the mutated samples inserted into the training datasets.

Table VII reports promising results, such that after augmentation, attack success rates are notably reduced in all settings. Retraining with more data is seen as more effective to reduce the attack success rates; more importantly, the model accuracy on hold-out test datasets is mostly consistent, if not better, than that of before. We thus interpret that our testing pipeline can be adopted by developers to enhance the robustness of their DNN-based code matching tools, by "repairing" potential mispredictions toward adversarial programs in the wild.

VII. Discussion

Comparison with Binary Rewriting Techniques. Conventional binary binary rewriting techniques are usually implemented as patch-based rewriting or duplication-based rewriting [23], [66]. We view that two key limitations impede their usage in our scenario. First, these methods can usually incur a high cost performance cost due to the extra control-flow transfers or even duplicated code sections introduced by rewriting. Second, it is often challenging, if at all possible, to *iteratively* mutate an already-rewritten binary code as such rewriting methods may likely introduce cumbersome structures in binary code (e.g., replica [66]). In contrast, our solution introduces negligible cost (as shown in Sec. VI-D). We iteratively mutate a perturbed binary code (Alg. 1) to gradually explore an mutation sequence that can deceive DNN models. Also, some other binary rewriting techniques, e.g., in-place rewriting [46], is often lightweight and cannot change the control-flow structures of binary code.

Comparison with Software Obfuscation Techniques. In addition to some graph-level perturbations that has been introduced in Sec. III-C, we also observed that the security community has proposed some heavyweight and comprehensive methods that can largely change the representation of programs. For instance, some relatively heavy-weight program obfuscation methods could incorporate a process-level virtual machine; it converts an input program into bytecode formats that are customized for the embedded virtual machine [50], [1]. During runtime, each bytecode instruction is emulated by the embedded virtual machine. Such obfuscation strategies has been commercialized [5] and shown to effectively hamper similarity analysis with its complex execution model.

To defeat DNN-based similarity analysis, readers may wonder about the feasibility to directly use such aforementioned heavyweight software obfuscation methods which can extensively change the visual appearances of (malicious) software and therefore deceive similarity analyzers. However, we note that programs mutated by such heavyweight approaches, while may manifest low matching score compared with the reference inputs, are *not* applicable in our scenario. The reasons are three fold: ① Heavyweight software obfuscations (such as virtual machine-based obfuscation) can often contain special patterns or routines, which are indeed not difficult for recognition. ② Extensively obfuscating programs, while can largely reduce the code similarity, can often lead to very high performance penalty and are not desired. In contrast, this research forms a set of stealthy and low-cost perturbations toward executable files and effectively deceive DNN-based binary code matching tools. ③ From the implementation perspective, commonly-used software obfuscation tools (e.g., [34], [20]) require the availability of software source code, which is not applicable in our scenario since we assume to directly mutate program binary code.

Perturbations vs. Metamorphic Relations. This research proposes a set of semantics-preserving perturbations to generate adversarial software examples and exploit DNN-based binary code matching tools. In general, this approach is conceptually related to metamorphic testing [15], [51]. In that sense, the semantics-preserving transformations used here are metamorphic relations (MRs) which are a core component of MT and which are usually invariant properties of a target software. Adversarial examples exposed in this research can be deemed as "bug-triggering inputs" of the DNN models, as to how MT is widely used to test software, cybersecurity, and ML model bugs [32], [33], [17], [71], [44], [57], [49], [64], [16]. We leave it as a future work to enhance the proposed framework with software (metamorphic) testing techniques.

VIII. Related Work

A number of research works have proposed to synthesize adversarial malware samples and evading ML-based malware analysis. As discussed in Sec. III-A, to preserve the correct functionality of the malware sample in binary code formats, techniques proposed by previous works primarily change nonfunctional components of a malware executable [35], [36], [56], [13], such as appending a few bytes at the end of the malware sample or perturbing data bytes that would never be loaded into memory. It might not be inaccurate to assume that such perturbations have limited application scope, and they can be easily detected via static or dynamic analysis techniques. [53] makes progressive approach by leveraging "in-place code randomization" [46] to mutate executables. The leveraged in-place transformations, although preserving the functionality by design, may only transform certain code chunks with specific structures.

IX. Conclusion

Binary code matching serves as the core basis of many software reverse engineering, re-engineering, and security tasks. This research generates adversarial executable examples to deceive DNN-based binary matching. Our technique enables program-wide perturbation on CFGs, and evaluations show that the mutated executables can effectively deceive DNNs in both white-box and black-box settings. We further illustrate the feasibility of enhancing DNN model robustness by retraining with our mutated program samples.

Acknowledgements

This work was supported in part by a RGC ECS grant under the contract 26206520, a Bridge Gap Fund of TTC/HKUST (Project ID: BGF.003.2021), and CCF-Tencent Open Research Fund. We are grateful to the anonymous reviewers for their valuable comments.

REFERENCES

[1] Code Virtualizer: Total obfuscation against reverse engineering. http://oreans.com/codevirtualizer.php.

[2] Full Implementation Details of Each Transformation Scheme on x86 assembly code. https://anonymous.4open.science/r/adversarial-example-78C1/implementation.md.

[3] Hex-Rays Microcode API vs. Obfuscating Compiler. https://hex-rays.com/blog/hex-rays-microcode-api-vs-obfuscating-compiler/.

[4] Research Artifacts. https://github.com/wwkenwong/Deceiving-DNN-based-Binary-Matching.

[5] VMProtect software protection. http://vmpsoft.com.

[6] Mohammed Abuhamad, Tamer AbuHmed, Aziz Mohaisen, and DaeHun Nyang. Large-scale and language-oblivious code authorship identification. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 101–114, 2018.

[7] Shushan Arakelyan, Christophe Hauser, Erik Kline, and Aram Galstyan. Towards learning representations of binary executable files for security tasks. *arXiv preprint arXiv:2002.03388*, 2020.

[8] Erick Bauman, Zhiqiang Lin, and Kevin W Hamlen. Superset disassembly: Statically rewriting x86 binaries without heuristics. In *NDSS*, 2018.

[9] Tal Ben-Nun, Alice Shoshana Jakobovits, and Torsten Hoefler. Neural code comprehension: A learnable representation of code semantics. NIPS 2018, 2018.

[10] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.

[11] Aylin Caliskan-Islam, Richard Harang, Andrew Liu, Arvind Narayanan, Clare Voss, Fabian Yamaguchi, and Rachel Greenstadt. De-anonymizing programmers via code stylometry. In *24th {USENIX} Security Symposium ({USENIX} Security 15)*, pages 255–270, 2015.

[12] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 ieee symposium on security and privacy (sp)*, pages 39–57. IEEE, 2017.

[13] R. L. Castro, C. Schmitt, and G. D. Rodosek. ARMED: How automatic malware modifications can evade static detection? In *2019 5th International Conference on Information Management (ICIM)*, pages 20–27, 2019.

[14] Haibo Chen, Liwei Yuan, Xi Wu, Binyu Zang, Bo Huang, and Pen-chung Yew. Control flow obfuscation with information flow tracking. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO 42, pages 391–400, New York, NY, USA, 2009. ACM.

[15] Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. Metamorphic testing: a new approach for generating next test cases. Technical report, Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong ..., 1998.

[16] Tsong Yueh Chen, Fei-Ching Kuo, Wenjuan Ma, Willy Susilo, Dave Towey, Jeffrey Voas, and Zhi Quan Zhou. Metamorphic testing for cybersecurity. *Computer*, 49(6):48–55, 2016.

[17] Tsong Yueh Chen, TH Tse, and Zhi Quan Zhou. Semi-proving: An integrated method for program proving, testing, and debugging. *IEEE Transactions on software engineering*, 37(1):109–125, 2010.

[18] Yizheng Chen, Yacin Nadji, Athanasios Kountouras, Fabian Monrose, Roberto Perdisci, Manos Antonakakis, and Nikolaos Vasiloglou. Practical attacks against graph-based clustering. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1125–1142, 2017.

[19] Moustapha Cisse, Yossi Adi, Natalia Neverova, and Joseph Keshet. Houdini: Fooling deep structured prediction models. *arXiv preprint arXiv:1707.05373*, 2017.

[20] Christian Collberg. The Tigress diversifying C virtualizer. http://tigress.cs.arizona.edu.

[21] Christian Collberg, Clark Thomborson, and Douglas Low. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 184–196, 1998.

[22] Hanjun Dai, Hui Li, Tian Tian, Xin Huang, Lin Wang, Jun Zhu, and Le Song. Adversarial attack on graph structured data. *arXiv preprint arXiv:1806.02371*, 2018.

[23] Zhui Deng, Xiangyu Zhang, and Dongyan Xu. BISTRO: Binary component extraction and embedding for software security applications. In *Proceedings of the 18th European Symposium on Research in Computer Security (ESORICS '13)*, 2013.

[24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.

[25] S. H. Ding, B. M. Fung, and P. Charland. Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization. In *IEEE S&P*, 2019.

[26] Thomas Dullien and Rolf Rolles. Graph-based comparison of executable objects. *SSTIC*, 2005.

[27] Mark Gabel, Lingxiao Jiang, and Zhendong Su. Scalable detection of semantic clones. In *Proceedings of the 30th International Conference on Software Engineering*, ICSE '08, pages 321–330. ACM, 2008.

[28] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *ICML*, 2017.

[29] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[30] SA Hex-Rays. IDA Pro: a cross-platform multi-processor disassembler and debugger, 2014.

[31] Yikun Hu, Hui Wang, Yuanyuan Zhang, Bodong Li, and Dawu Gu. A semantics-based hybrid approach on binary code similarity comparison. *IEEE Transactions on Software Engineering*, 2019.

[32] Mingyue Jiang, Tsong Yueh Chen, Fei-Ching Kuo, Dave Towey, and Zuohua Ding. A metamorphic testing approach for supporting program repair without the need for a test oracle. *Journal of systems and software*, 126:127–140, 2017.

[33] Mingyue Jiang, Tsong Yueh Chen, Zhi Quan Zhou, and Zuohua Ding. Input test suites for program repair: A novel construction method based on metamorphic relations. *IEEE Transactions on Reliability*, 2020.

[34] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. Obfuscator-LLVM – software protection for the masses. In Brecht Wyseur, editor, *Proceedings of the IEEE/ACM 1st International Workshop on Software Protection, SPRO'15, Firenze, Italy, May 19th, 2015*, pages 3–9. IEEE, 2015.

[35] B. Kolosnjaji, A. Demontis, B. Biggio, D. Maiorca, G. Giacinto, C. Eckert, and F. Roli. Adversarial malware binaries: Evading deep learning for malware detection in executables. In *2018 26th European Signal Processing Conference*, pages 533–537, 2018.

[36] Felix Kreuk, Assi Barak, Shir Aviv-Reuven, Moran Baruch, Benny Pinkas, and Joseph Keshet. Adversarial examples on discrete sequences for beating whole-binary malware detection. *CoRR*, abs/1802.04528, 2018.

[37] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial machine learning at scale. *arXiv preprint arXiv:1611.01236*, 2016.

[38] J. Křoustek and P. Matula. Retdec: An open-source machine-code decompiler. [talk], July 2018. Presented at Pass the SALT 2018, Lille, FR.

[39] Per Larsen, Andrei Homescu, Stefan Brunthaler, and Michael Franz. SoK: Automated software diversity. In *IEEE S&P*, 2014.

[40] Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196, 2014.

[41] Xuezixiang Li, Qu Yu, and Heng Yin. PalmTree: Learning an assembly language model for instruction embedding. 2021.

[42] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. Graph matching networks for learning the similarity of graph structured objects. *arXiv preprint arXiv:1904.12787*, 2019.

[43] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493*, 2015.

[44] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. How effectively does metamorphic testing alleviate the oracle problem? *IEEE Transactions on software engineering*, 40(1):4–22, 2013.

[45] Robert Lyda and James Hamrock. Using entropy analysis to find encrypted and packed malware. *IEEE Security & Privacy*, 5(2):40–45, 2007.

[46] Vasilis Pappas, Michalis Polychronakis, and Angelos D Keromytis. Smashing the gadgets: Hindering return-oriented programming using in-place code randomization. In *Security and Privacy (S&P), 2012 IEEE Symposium on*, pages 601–615. IEEE, 2012.

[47] Matteo Piano. Infrastructure for Reassembleable Disassembling and Transformation. https://github.com/piax93/uroboros, 2018.

[48] Igor V. Popov, Saumya K. Debray, and Gregory R. Andrews. Binary obfuscation using signals. In *Proceedings of 16th USENIX Security Symposium on USENIX Security Symposium*, pages 19:1–19:16, Berkeley, CA, USA, 2007. USENIX Association.

[49] Kun Qiu, Zheng Zheng, Tsong Chen, and Pak-Lok Poon. Theoretical and empirical analyses of the effectiveness of metamorphic relation composition. *IEEE Transactions on software engineering*, 2020.

[50] Rolf Rolles. Unpacking virtualization obfuscators. In *Proceedings of the 3rd USENIX Conference on Offensive Technologies*, WOOT'09, 2009.

[51] Sergio Segura, Gordon Fraser, Ana B Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on software engineering*, 42(9):805–824, 2016.

[52] Mahmood Sharif, Keane Lucas, Lujo Bauer, Michael K Reiter, and Saurabh Shintre. Optimization-guided binary diversification to mislead neural networks for malware detection. *arXiv preprint arXiv:1912.09064*, 2019.

[53] Mahmood Sharif, Keane Lucas, Lujo Bauer, Michael K. Reiter, and Saurabh Shintre. Optimization-guided binary diversification to mislead neural networks for malware detection, 2019.

[54] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE S&P*, 2016.

[55] Nedim Šrndic and Pavel Laskov. Practical evasion of a learning-based classifier: A case study. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*, pages 197–211, 2014.

[56] Octavian Suciu, Scott E. Coull, and Jeffrey Johns. Exploring adversarial examples in malware detection. *CoRR*, abs/1810.08280, 2018.

[57] Chang-Ai Sun, An Fu, Pak-Lok Poon, Xiaoyuan Xie, Huai Liu, and Tsong Yueh Chen. METRIC+: a metamorphic relation identification technique based on input plus output domains. *IEEE Transactions on Software Engineering*, 2019.

[58] Yiwei Sun, Suhang Wang, Xianfeng Tang, Tsung-Yu Hsieh, and Vasant Honavar. Adversarial attacks on graph neural networks via node injections: A hierarchical reinforcement learning approach. In *Proceedings of The Web Conference 2020*, pages 673–683, 2020.

[59] Yiwei Sun, Suhang Wang, Xianfeng Tang, Tsung-Yu Hsieh, and Vasant Honavar. Non-target-specific node injection attacks on graph neural networks: A hierarchical reinforcement learning approach. In *Proc. WWW*, volume 3, 2020.

[60] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

[61] Binghui Wang and Neil Zhenqiang Gong. Attacking graph-based classification via manipulating the graph structure. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2023–2040, 2019.

[62] Ruoyu Wang, Yan Shoshitaishvili, Antonio Bianchi, Aravind Machiry, John Grosen, Paul Grosen, Christopher Kruegel, and Giovanni Vigna. Ramblr: Making reassembly great again. In *NDSS*, 2017.

[63] Shen Wang, Zhengzhang Chen, Xiao Yu, Ding Li, Jingchao Ni, Lu-An Tang, Jiaping Gui, Zhichun Li, Haifeng Chen, and Philip S Yu. Heterogeneous graph matching networks for unknown malware detection.

[73] Zeping Yu, Rui Cao, Qiyi Tang, Sen Nie, Junzhou Huang, and Shi Wu. Order matters: Semantic-aware neural networks for binary code

In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pages 3762–3770. AAAI Press, 2019.

[64] Shuai Wang and Zhendong Su. Metamorphic testing for object detection systems. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1053–1065, 2020.

[65] Shuai Wang, Pei Wang, and Dinghao Wu. Reassembleable disassembling. In *USENIX Security*, 2015.

[66] Shuai Wang, Pei Wang, and Dinghao Wu. UROBOROS: Instrumenting stripped binaries with static reassembling. In *SANER*, 2016.

[67] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. Detecting code clones with graph neural networkand flow-augmented abstract syntax tree. *arXiv preprint arXiv:2002.08653*, 2020.

[68] Marcin Waniek, Kai Zhou, Yevgeniy Vorobeychik, Esteban Moro, Tomasz P Michalak, and Talal Rahwan. Attack tolerance of link prediction algorithms: How to hide your relations in a social network. *arXiv preprint arXiv:1809.00152*, 2018.

[69] Jason Wei and Kai Zou. Eda: Easy data augmentation techniques for boosting performance on text classification tasks. *arXiv preprint arXiv:1901.11196*, 2019.

[70] Zhenyu Wu, Steven Gianvecchio, Mengjun Xie, and Haining Wang. Mimimorphism: A new approach to binary code obfuscation. In *Proceedings of the 17th ACM conference on Computer and communications security*, pages 536–546, 2010.

[71] Xiaoyuan Xie, Joshua WK Ho, Christian Murphy, Gail Kaiser, Baowen Xu, and Tsong Yueh Chen. Testing and validating machine learning classifiers by metamorphic testing. *Journal of Systems and Software*, 84(4):544–558, 2011.

[72] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. Neural network-based graph embedding for cross-platform binary code similarity detection. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pages 363–376. ACM, 2017.
similarity detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1145–1152, 2020.

[74] Zeping Yu, Wenxin Zheng, Jiaqi Wang, Qiyi Tang, Sen Nie, and Shi Wu. CodeCMR: Cross-modal retrieval for function-level binary source code matching. *Advances in Neural Information Processing Systems*, 33, 2020.

[75] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28:649–657, 2015.

[76] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2847–2856, 2018.